

LeNet-5 trên CIFAR-10

Assignment 4 · Kiến trúc kinh điển gặp dữ liệu khó · Scratch NumPy · TensorFlow/Keras · PyTorch

Báo cáo cho notebook `07_cifar10_lenet.ipynb`. Notebook `03_cifar10.ipynb` với kiến trúc `SmallCNN` **giữ nguyên, không sửa** — đây là lần chạy song song để so sánh.

Mọi con số lấy từ `results/07_cifar10_lenet.json` và `results/03_cifar10.json`.

1. Vì sao thử LeNet-5 trên CIFAR-10

1.1 Câu hỏi của thí nghiệm

LeNet-5 (LeCun và cộng sự, 1998) được thiết kế cho ảnh **32×32 thang xám** chứa chữ số viết tay. CIFAR-10 là ảnh **32×32 màu** chụp mười loại vật thể thật: máy bay, ô tô, chim, mèo, hươu, chó, ếch, ngựa, tàu, xe tải.

Kích thước đầu vào trùng khít. Độ khó thì không.

Đó chính là điều khiến cặp dataset này đáng thử. Giữ nguyên kiến trúc, chỉ đổi độ khó của dữ liệu, để cô lập một câu hỏi duy nhất:

Một kiến trúc năm 1998 đi được bao xa trên bài toán nó chưa từng được thiết kế cho?

Báo cáo `08_lenet_mnist_report.pdf` cho thấy trên MNIST, LeNet-5 hoà với kiến trúc baseline. Báo cáo này kiểm tra xem điều đó có còn đúng khi dữ liệu khó lên hay không.

1.2 Kiến trúc

```
input 3×32×32
↓
C1 Conv 6 @ 5×5, pad=0 → 6×28×28 feature maps
↓ ReLU
S2 MaxPool 2×2 → 6×14×14 subsample
↓
C3 Conv 16 @ 5×5, pad=0 → 16×10×10 feature maps
↓ ReLU
S4 MaxPool 2×2 → 16×5×5 subsample
↓ Flatten → 400
F5 Dense 400 → 120 ↓ ReLU
F6 Dense 120 → 84 ↓ ReLU
out Dense 84 → 10 (logits)
```

Khác với bản MNIST đúng **một chỗ**: C1 ở đây dùng `pad=0`. CIFAR-10 vốn đã là 32×32 — đúng kích thước LeNet-5 được viết ra để nhận — nên convolution *valid* đưa 32×32 xuống 28×28 y như bài báo gốc. Từ S2 trở đi, hai mạng giống hệt nhau đến từng con số.

1.3 Phân bổ tham số

Lớp	Tham số	Tỷ lệ
C1 Conv 3→6, 5×5	456	0.7%
C3 Conv 6→16, 5×5	2,416	3.9%
F5 Linear 400→120	48,120	77.6%
F6 Linear 120→84	10,164	16.4%
out Linear 84→10	850	1.4%
Tổng	62,006	100%

Con số cần ghi nhớ khi đọc phần kết quả: **bộ trích xuất đặc trưng chỉ có 2,872 tham số** — 4.6% toàn mạng. Sáu bộ lọc ở C1 và mười sáu ở C3 là toàn bộ vốn từ thị giác mà mạng này có để mô tả mười loại vật thể chụp ở đủ tư thế, ánh sáng và nền.

So sánh trước: `SmallCNN` trong `03_cifar10.ipynb` có **545,098** tham số, dùng 32 và 64 kênh. LeNet-5 ở đây nhẹ hơn khoảng **8.8 lần**.

2. "Modernized" nghĩa là gì

Bản LeNet-5 dùng ở đây **không phải bản 1998 nguyên văn**:

Thành phần	LeNet-5 gốc (1998)	Bản dùng ở đây
Hàm kích hoạt	<code>tanh</code>	ReLU
Lớp subsample	Average pooling	Max pooling
Topology	6→16 maps, 5×5, head 120→84→10	giữ nguyên
Optimizer	SGD	Adam (khớp baseline)

Lý do kỹ thuật: `scratch_nn.py` **chỉ hiện thực ReLU và MaxPool2D** — không có `Tanh`, không có `AvgPool2D`. Bản modernized cho phép cả ba framework chạy **cùng một mạng** mà không phải viết thêm đạo hàm ngược cho hai loại lớp mới. Đây cũng là bản mà hầu hết implementation hiện đại dùng.

3. Thiết lập thí nghiệm

Cùng dataset + cùng split + cùng hyperparameters — chỉ đổi kiến trúc.

Thiết lập	Giá trị
Subset so sánh 3 framework	5,000 train / 2,000 test
Tham chiếu full-data	50,000 train / 10,000 test
Epochs	5
Batch size	64
Optimizer	Adam, lr = 1e-3
Seed	42 (<code>U.set_seed</code> gọi lại trước mỗi framework)
Chuẩn hoá	Per-channel, thống kê tính từ tập train

Mọi thiết lập khớp chính xác với `03_cifar10.ipynb`, nên chênh lệch kết quả chỉ có thể đến từ kiến trúc.

3.1 Lưu ý bắt buộc khi đọc cột thời gian

PyTorch chạy GPU, TensorFlow chạy CPU. TensorFlow từ bản 2.11 không còn build GPU cho Windows native; máy chạy bài có RTX 3060 Laptop nhưng chỉ PyTorch dùng được.

Cột `train_seconds` vì thế **so sánh hai loại phần cứng khác nhau**, không phản ánh chất lượng framework. Cột `n_params` và `test_accuracy` thì so sánh được trực tiếp.

3.2 Ba cách hiện thực cùng một mạng

Thành phần	Scratch (NumPy)	TensorFlow/Keras	PyTorch
Định nghĩa mạng	<code>S.Sequential([...])</code>	<code>keras.Sequential([...])</code>	<code>nn.Module (LeNet5)</code>
C1 / C3	<code>S.Conv2D(k=5) im2col + matmul</code>	<code>layers.Conv2D(5)</code>	<code>nn.Conv2d(5)</code>
S2 / S4	<code>S.MaxPool2D</code> định tuyến argmax	<code>layers.MaxPooling2D</code>	<code>nn.MaxPool2d</code>
F5 / F6 / out	<code>S.Dense (x@W+b)</code>	<code>layers.Dense</code>	<code>nn.Linear</code>
Gradient	đạo hàm viết tay + col2im	tự động (GradientTape)	tự động (autograd)
Vòng huấn luyện	for-loop tường minh	<code>model.fit()</code>	for-loop tường minh
Phần cứng	CPU	CPU	GPU (cuDNN)

3.3 Định nghĩa mô hình — cùng một mạng, ba cách viết

Ba đoạn dưới đây trích nguyên văn từ `07_cifar10_lenet.ipynb`. Cả ba dựng đúng cùng một kiến trúc và phải ra cùng một số tham số — notebook có `assert` kiểm tra điều đó ngay sau mỗi đoạn.

A — Scratch (NumPy). Mọi lớp đến từ `scratch_nn.py`, nơi forward và backward đều viết tay. Chú thích `# C1`, `# S2` ... bám đúng ký hiệu bài báo 1998.

```
U.set_seed(U.SEED)

def build_scratch_lenet():
    return S.Sequential([
        S.Conv2D(C, C1_OUT, k=KERN, stride=1, pad=PAD1, seed=1),    # C1
        S.ReLU(),
        S.MaxPool2D(2, 2),                                          # S2
        S.Conv2D(C1_OUT, C3_OUT, k=KERN, stride=1, pad=PAD3, seed=2), # C3
        S.ReLU(),
        S.MaxPool2D(2, 2),                                          # S4
        S.Flatten(),
        S.Dense(FLAT, F5_OUT, seed=3),                               # F5
        S.ReLU(),
        S.Dense(F5_OUT, F6_OUT, seed=4),                             # F6
        S.ReLU(),
        S.Dense(F6_OUT, N_CLASSES, seed=5),                         # output
    ])

scratch_model = build_scratch_lenet()
```

B — TensorFlow / Keras. Khai báo thay vì tự dựng. Hai tham số `padding` tái hiện đúng cặp `pad` dùng ở bản scratch.

```
U.set_seed(U.SEED)

# channels-first -> channels-last for Keras
Xtr_k = np.transpose(X_train, (0, 2, 3, 1))
Xte_k = np.transpose(X_test, (0, 2, 3, 1))
print("keras input shape:", Xtr_k.shape)

def build_keras_lenet(name):
    return keras.Sequential([
        keras.layers.Input(shape=(H, W, C)),
        keras.layers.Conv2D(C1_OUT, KERN, padding="valid", activation="relu"),
        keras.layers.MaxPooling2D(2),
        keras.layers.Conv2D(C3_OUT, KERN, padding="valid", activation="relu"),
        keras.layers.MaxPooling2D(2),
        keras.layers.Flatten(),
        keras.layers.Dense(F5_OUT, activation="relu"),
        keras.layers.Dense(F6_OUT, activation="relu"),
        keras.layers.Dense(N_CLASSES),
    ], name=name)

keras_model = build_keras_lenet("lenet5")
keras_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LR),
    loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
```

C — PyTorch. Tách `features` (C1–S4) và `classifier` (F5–output) để hai nửa của LeNet hiện rõ trong cấu trúc lớp.

```

U.set_seed(U.SEED)

class LeNet5(nn.Module):
    """LeNet-5 (LeCun et al., 1998), modernized with ReLU + max pooling."""

    def __init__(self, in_ch, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, C1_OUT, KERN, padding=PAD1),      # C1
            nn.ReLU(),
            nn.MaxPool2d(2),                                   # S2
            nn.Conv2d(C1_OUT, C3_OUT, KERN, padding=PAD3),     # C3
            nn.ReLU(),
            nn.MaxPool2d(2),                                   # S4
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(FLAT, F5_OUT),                           # F5
            nn.ReLU(),
            nn.Linear(F5_OUT, F6_OUT),                         # F6
            nn.ReLU(),
            nn.Linear(F6_OUT, n_classes),                      # output
        )

    def forward(self, x):
        x = self.features(x)      # feature extraction
        x = self.classifier(x)    # classification
        return x

torch_model = LeNet5(C, N_CLASSES).to(DEVICE)
n_torch = sum(p.numel() for p in torch_model.parameters())

```

3.4 Vòng huấn luyện

A — Scratch. `S.fit` tự hiện thực vòng lặp: forward → loss → backward → optimizer step.

```

U.set_seed(U.SEED)
with U.Timer() as t_scratch:
    hist_scratch = S.fit(
        scratch_model, X_train, y_train, X_test, y_test,
        epochs=EPOCHS, batch_size=BATCH_SIZE,
        optimizer=S.Adam(LR), seed=U.SEED,
    )
print(f"\ntotal training time: {t_scratch.seconds:.1f}s")

```

B — Keras. Vòng lặp nằm bên trong `.fit()`, không nhìn thấy được.

```

U.set_seed(U.SEED)
with U.Timer() as t_keras:
    hk = keras_model.fit(
        Xtr_k, y_train, validation_data=(Xte_k, y_test),
        epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=2,
    )
print(f"\ntotal training time: {t_keras.seconds:.1f}s")

```

C — PyTorch. Vòng lặp viết tường minh, nên chu trình `zero_grad → forward → loss → backward → step` hiện nguyên trên mã nguồn.

```

def train_torch(model, Xtr, ytr, Xte, yte, epochs=EPOCHS, batch_size=BATCH_SIZE, lr=LR,
log=True):
    # Explicit loop: zero grad -> forward -> loss -> backward -> update.
    loader = DataLoader(TensorDataset(torch.tensor(Xtr), torch.tensor(ytr)),
                        batch_size=batch_size, shuffle=True)
    Xte_t = torch.tensor(Xte).to(DEVICE)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_acc": []}

    with U.Timer() as t:
        for epoch in range(1, epochs + 1):
            model.train()
            running, nb = 0.0, 0
            for xb, yb in loader:
                xb, yb = xb.to(DEVICE), yb.to(DEVICE)

                optimizer.zero_grad()
                logits = model(xb)
                loss = criterion(logits, yb)
                loss.backward()
                optimizer.step()

                running += loss.item(); nb += 1

            model.eval()
            with torch.no_grad():
                preds = []
                for i in range(0, len(Xte_t), 1024):
                    preds.append(model(Xte_t[i:i + 1024]).argmax(1).cpu().numpy())
                acc = float((np.concatenate(preds) == yte).mean())

            history["loss"].append(running / nb)
            history["val_acc"].append(acc)
            if log:
                print(f"epoch {epoch}/{epochs}  loss {running/nb:.4f}  test_acc {acc:.4f}")

    return history, t.seconds

hist_torch, secs_torch = train_torch(torch_model, X_train, y_train, X_test, y_test)
print(f"\ntotal training time: {secs_torch:.1f}s")

```

Ba cách này cho cùng một thuật toán. Khác biệt chỉ nằm ở chỗ **bao nhiêu phần của vòng lặp được giấu đi**: Keras giấu hết, PyTorch giấu mỗi phần tính gradient, bản scratch không giấu gì.

4. Kết quả

4.1 Ba framework trên subset 5k

Framework	Tham số	Accuracy	Macro-F1	Train (s)	Phần cứng
TensorFlow/Keras	62,006	45.45%	0.4528	4.7	CPU
PyTorch	62,006	44.60%	0.4363	1.5	GPU
Scratch (NumPy)	62,006	42.70%	0.4216	28.7	CPU

Ba framework ra **đúng 62,006 tham số**, khớp với tính tay ở mục 1.3 — xác nhận kiến trúc được dịch đúng sang cả ba nơi.

Chênh lệch accuracy giữa ba bản là 2.75 điểm, rộng hơn hẳn mức 0.20 điểm quan sát được trên MNIST. Bài toán càng khó, cùng một mạng càng nhạy với khởi tạo và thứ tự shuffle.

4.2 Full-data (50k)

Framework	Accuracy	Macro-F1	Train (s)
PyTorch	60.75%	0.6070	30.9
TensorFlow/Keras	60.32%	0.5985	29.3

Gấp 10 lần dữ liệu đổi lấy **+16.15 điểm** (44.60% → 60.75%). So với MNIST, nơi cùng phép tăng dữ liệu chỉ cho khoảng +1 điểm, con số này cho thấy CIFAR-10 còn rất xa mức bão hoà.

5. LeNet-5 so với SmallCNN

03_cifar10.ipynb dùng SmallCNN : hai lớp conv 3×3 với 32 rồi 64 kênh, sau đó Linear(4096 → 128) → Linear(128 → 10) . Tổng 545,098 tham số — **gấp 8.79 lần** LeNet-5.

5.1 Bảng đối đầu

Dataset	Framework	SmallCNN	LeNet-5	Chênh
CIFAR-10 (5k subset)	Scratch (NumPy)	50.55%	42.70%	−7.85
CIFAR-10 (5k subset)	TensorFlow/Keras	54.85%	45.45%	−9.40
CIFAR-10 (5k subset)	PyTorch	53.10%	44.60%	−8.50
CIFAR-10 (full 50k)	TensorFlow/Keras	70.43%	60.32%	−10.11
CIFAR-10 (full 50k)	PyTorch	71.90%	60.75%	−11.15

Chênh lệch trung bình: −9.40 điểm. LeNet-5 thua ở **cả năm** phép so sánh.

Đây là kết quả trái ngược hẳn với MNIST, nơi chênh lệch trung bình chỉ −0.22 điểm và hai kiến trúc thực chất là hoà.

5.2 Phát hiện quan trọng nhất — khoảng cách rộng ra khi có thêm dữ liệu

Lượng dữ liệu	Chênh lệch trung bình
5k subset	−8.58 điểm
full 50k	−10.63 điểm

Trực giác thông thường nói ngược lại: thêm dữ liệu thì mô hình yếu sẽ đuổi kịp. Ở đây điều ngược lại xảy ra — dữ liệu gấp 10 lần làm khoảng cách **rộng thêm 2 điểm**.

Nhìn theo mức hưởng lợi từ dữ liệu:

LeNet-5	44.60%	→	60.75%	+16.15 điểm
SmallCNN	53.10%	→	71.90%	+18.80 điểm
SmallCNN tận dụng dữ liệu tốt hơn				+2.65 điểm

Đây là dấu hiệu kinh điển của **nghẽn dung lượng (capacity bottleneck)**. Khi mô hình không đủ chỗ để biểu diễn những gì dữ liệu chứa, việc đưa thêm dữ liệu vào chỉ giúp được tới một mức rồi dừng. Dữ liệu mới mang thông tin mà mạng **không có chỗ để lưu**.

Rút ra: thêm dữ liệu không sửa được mạng quá nhỏ. Hai thứ đó không thay thế cho nhau.

5.3 Nghẽn ở đâu — nhìn vào số kênh

Vấn đề không nằm ở tổng số tham số mà ở **phần nào giữ số tham số đó**:

	Bộ trích xuất đặc trưng	Bộ phân loại	Tổng
LeNet-5	2,872 (4.6%)	59,134	62,006
SmallCNN	19,392 (3.6%)	525,706	545,098

LeNet-5 có **6 rồi 16 bộ lọc**. SmallCNN có **32 rồi 64**. Với ảnh xám chứa chữ số, sáu loại nét cơ bản là đủ — nét ngang, nét dọc, vài đường cong. Với ảnh màu chụp mèo, tàu thủy và máy bay ở đủ tư thế, sáu bộ lọc ở tầng đầu đơn giản là **không đủ vốn từ thị giác**.

Mọi thứ xảy ra sau C1 đều bị giới hạn bởi những gì C1 giữ lại được. Đó là nghẽn cổ chai thật sự, và không lượng huấn luyện nào chữa được.

5.4 Rẻ hơn về tính toán, nhưng đó không phải điều đang được hỏi

	Tham số	Conv MACs	Scratch train (s)
SmallCNN	545,098	5,603,328	83.0
LeNet-5	62,006	592,800	28.7

LeNet-5 rẻ hơn 9.45 lần về phép tính conv và chạy nhanh hơn 2.89 lần ở bản NumPy.

Nếu bài toán là chạy trên thiết bị nhúng, đó là ưu thế thật. Nhưng ở đây câu hỏi là accuracy, và LeNet trả giá 9.4 điểm để đổi lấy khoản tiết kiệm đó. **Trên CIFAR-10 thì đó là một đánh đổi tồi.**

5.5 Mạng nhằm lẫn ở đâu

Recall theo từng lớp, lần chạy tốt nhất (PyTorch, full 50k, 60.75%):

Lớp	Recall		Lớp	Recall
frog	0.766		horse	0.618
ship	0.740		cat	0.539
automobile	0.691		deer	0.502
truck	0.690		bird	0.464
airplane	0.677		dog	0.388

Cặp nhầm nặng nhất: **dog → cat, 322 ảnh**.

Mô hình phân biệt tốt các lớp có **hình bao và nền đặc trưng** — ếch (nền xanh đồng nhất), tàu (nền nước), xe cộ (đường nét thẳng, bánh xe). Nó sụp ở các lớp **động vật bốn chân có lông**, nơi cần kết cấu tinh tế để tách chó khỏi mèo khỏi hươu.

Điều này khớp chính xác với chẩn đoán ở mục 5.3: sáu bộ lọc 5×5 đủ để bắt đường bao và mảng màu lớn, nhưng không đủ để mã hoá khác biệt kết cấu giữa lông chó và lông mèo.

6. Kết luận

LeNet-5 thua rõ ràng trên CIFAR-10: −9.40 điểm trung bình, thua ở cả năm phép so sánh, và khoảng cách rộng thêm khi có nhiều dữ liệu hơn.

Điều đó **không** có nghĩa LeNet-5 là kiến trúc tồi. Cùng mạng này hoà với baseline trên MNIST (xem `08_lenet_mnist_report.pdf`). Kết luận đúng là:

Dung lượng kiến trúc phải tương xứng với độ khó của dữ liệu.

Ba điều đáng mang đi:

- 1. **Cùng kích thước đầu vào không có nghĩa cùng độ khó.** MNIST và CIFAR-10 đều là ảnh 32×32. Một bên LeNet-5 hoà, bên kia thua 9.4 điểm. Kích thước tensor không nói gì về độ phức tạp thị giác.
- 2. **Thêm dữ liệu không cứu được mạng quá nhỏ.** Dữ liệu gấp 10 lần làm khoảng cách rộng ra chứ không hẹp lại. Khi đã nghẽn dung lượng, phải mở rộng mô hình — không phải thu thập thêm dữ liệu.
- 3. **Nghẽn nằm ở số kênh, không ở tổng tham số.** LeNet-5 giữ 95% tham số trong bộ phân loại và chỉ 2,872 trong phần convolution. Sáu bộ lọc ở tầng đầu là trần thực sự của mạng này.

6.1 Liên hệ với thí nghiệm M1→M4

`04_compare.ipynb` cải tiến CNN theo một trục khác: giữ ngân sách tham số gần như cố định và thay đổi **cơ chế** (BatchNorm, residual, SE attention).

Hai trục này độc lập với nhau:

Trục	Câu hỏi	Ví dụ trong bài
Dung lượng	Mạng có đủ chỗ để biểu diễn không?	LeNet-5 (62k) vs SmallCNN (545k)
Cơ chế	Mạng có học hiệu quả phần chỗ nó có không?	M1 → M2 → M3 → M4

Báo cáo này đo trục thứ nhất. Khi dung lượng là thứ đang thiếu, không cơ chế nào bù được.

6.2 Hạn chế

Kết quả dựa trên **một seed, 5 epochs, không tinh chỉnh hyperparameter, không data augmentation**. Con số tuyệt đối (60.75%) thấp hơn nhiều so với mức CIFAR-10 hiện đại (>95%) và không nhằm cạnh tranh — chúng chỉ nhằm so sánh công bằng giữa hai kiến trúc dưới cùng điều kiện.

Chênh lệch −9.40 điểm thì đủ lớn để vượt xa vùng nhiễu, nên kết luận về hướng là chắc chắn; riêng độ lớn chính xác thì nên hiểu là xấp xỉ.